



METRIC 3D MODEL GENERATOR

A PROJECT REPORT

for

U23AM491 – MACHINE LEARNING

Submitted by

NAVIYA DHARSHINI A S 722823243083

in partial fulfillment for the award of the

degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SRI ESHWAR COLLEGE OF ENGINEERING

(An Autonomous Institution, Affiliated to Anna University - Chennai - 600 025)

NOVEMBER / DECEMBER 2025

DECLARATION

I declare that the work entitled “**METRIC 3D MODEL GENERATOR**” is submitted in partial fulfillment of the requirement for the award of the degree in B. Tech. (AI&DS), Anna University Chennai, is a record of the my own work carried out by me during the academic year 2025-2026 under the supervision and guidance of **Dr. G. Sathish Kumar**, Associate Professor, Department of Artificial Intelligence and Data Science, Sri Eshwar College of Engineering, Coimbatore. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate places. The matter embodied in this work is original and has not been submitted for the award of any other degree or diploma, either in this or any other University.

Student Name

Register Number

Signature

Naviya Dharshini A S

722823243083

I certify that the declaration made above by the candidate is true.

Dr. G. Sathish Kumar

Associate Professor

Department of AI&DS



BONAFIDE CERTIFICATE

Certified that this project report “**METRIC 3D MODEL GENERATOR**” is the bonafide work of “**Naviya Dharshini A S (722823243083)**” who carried out the project work under my supervision.

SIGNATURE

Dr. M. MOHAMMED MUSTAFA

Head of the Department

Associate Professor & Head

Dept. of Artificial Intelligence and Data Science

Science Sri Eshwar College of Engineering, Coimbatore

Coimbatore

SIGNATURE

Dr. G. SATHISH KUMAR

Supervisor

Associate Professor

Dept. of Artificial Intelligence and Data

Sri Eshwar College of Engineering,

Submitted for the Project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I thank the Almighty for giving me an opportunity to study in a prestigious institution and grace to complete this project successfully.

I also express my gratitude to our Chairperson **Mr. R. Mohanram** and Director **Mr. R. Rajaram**, Sri Eshwar College of Engineering for their endeavor to provide all the facilities we require and the interest they showed in the welfare of the students.

I wish to express my sincere thanks to **Dr. Sudha Mohanram**, the **Principal & Joint Managing Trustee** of our college for providing the necessary infrastructure facilities.

I express my sincere gratitude to **Dr. M. Mohammed Mustafa** the **Head of the Department**, Artificial Intelligence and Data Science, for his guidance and providing valuable suggestion and all necessary facilities at the right time for doing this project.

I express my gratitude and sincere thanks to my guide **Dr. G. Sathish Kumar**, **Associate Professor**, Dept. of Artificial Intelligence and Data Science, for his valuable suggestions and constant encouragement for successful completion of this project.

I would also extend my hearty thanks to my family members and all the friends who encouraged me to do this project successfully.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	VII
	LIST OF FIGURES	I
	LIST OF ABBREVIATIONS	IX
1.	INTRODUCTION	1
2.	LITERATURE REVIEW	2
3.	PROBLEM DEFINITION	4
	3.1 EXISTING SYSTEM	4
	3.2 DISADVANTAGES OF EXISTING SYSTEM	
	3.3 PROPOSED SYSTEM	7
	3.4 ADVANTAGES OF PROPOSED SYSTEM	8
4.	SYSTEM STUDY	9
	4.1 FEASIBILITY STUDY	9
	4.1.1 Economical Feasibility	9
	4.1.2 Technical Feasibility	10
	4.1.3 Social Feasibility	10

5.	SYSTEM REQUIREMENTS SPECIFICATION	11
	5.1 HARDWARE REQUIREMENTS	11
	5.2 SOFTWARE REQUIREMENTS	11
	5.3 SOFTWARE DESCRIPTION	12
6.	SYSTEM DESIGN	16
	6.1 SYSTEM ARCHITECTURE	16
	6.2 DATA FLOW DIAGRAM	16
7.	SYSTEM IMPLEMENTATION	17
	7.1 MODULES	17
	7.2 MODULE DESCRIPTION	17
	7.2.1 IMAGE LOADING AND PREPROCESSING	17
	7.2.2 ONNX ROOM SEGMENTATION	18
	7.2.3 MASK TO POLYGON CONVERSION	18
	7.2.4 PLAN ESTIMATION	18
	7.2.5 3D MODEL GENERATION	19
	7.2.6 ERROR HANDLING	21

8.	MACHINE LEARNING MODEL DEVELOPMENT	21
	8.1 UNIT TESTING	21
	8.2 INTEGRATION TESTING	21
	8.3 FUNCTIONAL TESTING	21
	8.4 SYSTEM TESTING	21
	8.5 WHITE BOX TESTING	21
	8.6 BLACK BOX TESTING	21
	8.7 MAINTANCE	21
9.	FUTURE ENHANCEMENT AND CONCLUSION	24
10.	APPENDIX	26
	A.1 SOURCE CODE	26
	A.2 SCREENSHOTS	31
11.	REFERENCES	33

ABSTRACT

The 2D-to-3D Floor Plan Model Generator is an automated system designed to convert architectural floor plans into accurate 3D building models. Traditional workflows require architects and designers to manually trace walls, rooms, and openings in CAD tools or software like SketchUp and Blender, which is slow, repetitive, and error-prone. This project addresses that challenge by combining image processing, a deep-learning-based segmentation model, and geometric post-processing to extract structural elements from 2D plans and automatically generate a metric 3D representation.

The system accepts floor plans in image formats (PNG/JPG) and CAD formats (DXF). For image inputs, OpenCV-based preprocessing, resizing, padding, and color conversion are used to prepare the image for inference. A trained room-segmentation neural network, exported to ONNX format and executed with ONNX Runtime, predicts a binary mask that highlights room regions in the floor plan. This mask is converted into precise room polygons using contour extraction. For DXF inputs, the system reads CAD entities and layers directly and maps them into the same internal geometric representation. A scaling mechanism ensures that all extracted coordinates are converted into real-world metric units so that parameters like wall thickness, wall length, and floor height can be controlled accurately.

The backend pipeline, implemented in Python, parses and cleans the 2D geometry, runs the ONNX room-segmentation model, and applies heuristic estimators to derive high-level project metrics such as number of rooms, approximate wall counts, total wall length, total floor area, and estimated work days based on area. The final model is exported in standard formats such as GLB/GLTF and OBJ, enabling visualization in web-based 3D viewers or import into downstream design tools.

By automating room detection, geometry extraction, and 3D mesh generation, the system significantly reduces manual modelling effort while still allowing users to configure key parameters such as wall height, wall thickness, and scale. The solution is lightweight, built entirely on open-source tools such as OpenCV, ONNX Runtime, and Blender. Overall, the project demonstrates an effective integration of deep learning-based image segmentation, geometric reasoning, and 3D graphics automation to transform static floor plans into interactive 3D models, improving productivity in architectural visualization, interior design, construction planning, and educational applications.

LIST OF FIGURES

FIGURE	TITLE	PAGE
No.		No.
6.1	Model architecture	10
6.2	System workflow	12
6.3	Block Diagram	13
10.2.1	terminal output 1	47
10.2.2	terminal output 2	48

LIST OF ABBREVIATIONS

ML	Machine Learning
DXF	Drawing Exchange Format
GLB	ITF Binary Format
CLI	Command Line Interface
GUI	Graphical User Interface
OBJ	Wavefront Object Format
ONNX	Open Neural Network Exchange

CHAPTER 1

INTRODUCTION

Architectural floor plans are usually created as 2D drawings in CAD tools or exported as images. To visualize these designs in 3D, architects and students often need to manually recreate the same layout in 3D modelling software such as Blender or SketchUp. This process involves redrawing walls, extruding shapes, and adding openings, which is slow, repetitive, and prone to errors in dimensions or alignment. In academic environments, this manual work also limits how quickly different design alternatives can be explored.

The 2D-to-3D Floor Plan Model Generator aims to overcome these limitations by automatically converting 2D floor plans into accurate 3D building models. These polygons are then used to build a 3D representation of the floor plan with configurable parameters such as wall height, wall thickness, and global scale.

The project is implemented as a modular Python-based pipeline. First, the input image is loaded and preprocessed using OpenCV, including resizing, padding, and color conversion. The processed image is then passed to an ONNX room-segmentation model executed with ONNX Runtime, which predicts a binary mask of room areas. This mask is converted into polygons using contour extraction, and additional heuristics are applied to estimate global properties such as number of rooms, total wall length, floor area, and approximate work days required for construction. For DXF files, geometric entities and layers are read directly and mapped into the same internal representation.

In the final stage, a Blender automation script consumes the structured floor-plan description (walls, rooms, and parameters) and generates the 3D model using mesh extrusion and boolean operations. The output is exported as GLB/GLTF or OBJ, allowing users to open the result in web-based viewers or import it back into other 3D tools. The system is designed to be lightweight, relying only on open-source components such as Python, OpenCV, ONNX Runtime, and Blender, and can run on ordinary desktop hardware.

Overall, the project demonstrates how computer vision, deep learning segmentation, and 3D graphics automation to turn static architectural drawings into interactive 3D models. This reduces manual effort, speeds up design visualization, and provides an educational tool for students learning about building layouts, quantities, and basic construction estimation.

CHAPTER 2

LITERATURE REVIEW

Several research works and tools are related to automatic understanding of floor plans and 3D reconstruction. Many earlier systems rely mainly on classical image processing, such as edge detection and Hough transforms, to identify walls and line segments from raster floor plan images. These approaches work well on clean, high-contrast drawings but often struggle with noisy scans, varying line widths, or complex layouts, and they usually require significant manual post-editing in CAD software.

Recent work in deep learning-based floor plan analysis has shown that convolutional neural networks can be trained to segment semantic regions of floor plans, such as rooms, corridors, and openings. Segmentation networks output masks where each pixel is assigned to a specific class, which can then be converted into vector polygons. This is closely related to the approach used in the current project, where a trained room-segmentation model is exported to ONNX format and executed with ONNX Runtime to detect room regions from preprocessed images.

Other studies and commercial tools focus on floor plan to 3D reconstruction, where the detected geometry is extruded into 3D. Some systems are integrated directly into BIM or CAD platforms and rely on proprietary algorithms. Academic prototypes often demonstrate the feasibility of automatically generating 3D models but may require very constrained input formats or high-quality vector data. Many of these solutions do not expose a simple, scriptable pipeline suitable for lightweight educational use or experimentation.

In addition, there is research on automatic quantity take-off and construction estimation from building models. Heuristic or rule-based methods are often used to approximate wall lengths, floor areas, and material quantities once a clean geometric representation is available. The estimation module in this project follows a similar idea: after rooms and walls are obtained, simple geometric formulas (such as polygon area and edge length) are applied to compute total area, wall length, approximate number of rooms, and estimated work days.

CHAPTER 3

PROBLEM DEFINITION

3.1 EXISTING SYSTEM

In the existing workflow, architects, engineers, and students typically start with a 2D floor plan drawn on paper, exported as an image, or created in CAD software. To obtain a 3D visualization, they manually recreate the layout in modelling tools such as AutoCAD, SketchUp, or Blender. This involves importing the 2D drawing, scaling it, tracing walls with line or polyline tools, and then extruding these shapes into 3D walls and slabs. Doors and windows must be added one by one, and openings must be adjusted carefully to match the original design.

Even when specialized software is used to convert 2D plans to 3D, most tools either require strict input formats or offer only semi-automatic conversion. Users still need to clean the drawings, fix scale, and manually correct incorrectly detected elements. These systems are often closed, commercial, and not easily customizable for experimentation or academic projects.

3.2 DISADVANTAGES OF EXISTING SYSTEM

- Manual tracing and modelling of walls, doors, and windows is time-consuming and repetitive.
- Errors in scale or alignment can lead to inaccurate 3D models and misleading visualizations.
- Many existing tools are proprietary, expensive, and not easily extensible for research or student projects.
- Semi-automatic solutions still require significant manual correction for noisy or scanned floor plans.
- Existing workflows typically do not integrate a learning model that segments rooms automatically from raw images.
- Quantity estimation (room counts, wall length, total area, approximate work days) is rarely integrated into simple 2D-to-3D pipelines.

3.3 PROPOSED SYSTEM

The proposed 2D-to-3D Floor Plan Model Generator introduces a fully automated pipeline that starts from a floor plan image or DXF file and produces a metric 3D model along with high-level structural estimates. The system first loads and preprocesses the floor plan using OpenCV, performing resizing, padding, and color conversion. Using ONNX Runtime, the system generates a binary room mask, which is converted into room polygons through contour extraction.

For plans available in DXF format, geometric entities and layers are read directly and mapped into the same internal representation used for segmented rooms and walls. A scaling mechanism converts all coordinates into real-world metric units. A heuristic estimation module then analyzes the resulting geometry to approximate the number of rooms, number of wall segments, total wall length, total area in square meters, and estimated work days based on area.

In the final stage, a Blender automation script consumes the structured plan data and generates the 3D model by extruding walls, creating floors, and applying openings. The 3D model is exported as GLB/GLTF or OBJ for viewing in web-based viewers or importing into other 3D tools. The system is modular, scriptable, and uses only open-source components, making it suitable for educational use and further research.

3.4 ADVANTAGES OF PROPOSED SYSTEM

- Automates room detection from 2D plans using a deep-learning room-segmentation model (ONNX), reducing manual tracing effort.
- Converts segmentation masks into precise room polygons, enabling clean geometric reconstruction.
- Generates metric 3D models automatically in Blender with configurable wall height, wall thickness, and scale.
- Provides additional analytical output such as estimated number of rooms, wall length, total floor area, and approximate work days.
- Uses open-source tools (Python, OpenCV, ONNX Runtime, Blender), making the system economical and extensible.
- Supports both image-based and DXF-based floor plans, increasing flexibility in input formats.
- Offers a reusable, script-based pipeline that can be integrated into academic, research, or small-scale professional workflows.

CHAPTER 4

SYSTEM STUDY

4.1 FEASIBILITY STUDY

- A feasibility study assesses whether the proposed 2D-to-3D Floor Plan Model Generator can be successfully developed, deployed, and adopted given available technological, economic, and social factors. This evaluation confirms the system provides a practical and valuable solution for architectural visualization and educational use.

4.1.1 ECONOMICAL FEASIBILITY

- The project is cost-effective since it leverages free and open-source software components, including Python, OpenCV for image processing, ONNX Runtime for running the trained segmentation model, and Blender for 3D model generation. There is no need for specialized hardware beyond a standard desktop or laptop computer with a camera for input images. This keeps development and usage costs low for students, educators, and small firms.

4.1.2 TECHNICAL FEASIBILITY

- Technically, the system is feasible due to the availability of mature libraries and toolkits that support all required functions: image preprocessing, inference with ONNX models, geometric processing, and 3D mesh generation. The segmentation model can run efficiently on CPUs without needing a GPU, enabling broader compatibility. The Blender scripting API allows automation of 3D model creation without manual modeling. The system supports common file formats (image, DXF, GLB/GLTF, OBJ) facilitating integration with existing workflows.

4.1.3 SOCIAL FEASIBILITY

- Socially, the project is acceptable and beneficial as it reduces manual effort in converting 2D designs into 3D models, speeding up visualization for architects and learners. Its user-friendly design, open documentation, and offline-capable tools make

it accessible to students and professionals globally. The educational value of automatically linking floor plans to 3D representations supports learning and design iteration, improving engagement and understanding.

CHAPTER 5

SYSTEM REQUIREMENTS SPECIFICATION

5.1 Hardware Requirements

The hardware requirements define the minimum and recommended configurations needed to run the 2D-to-3D Floor Plan Model Generator smoothly, including ONNX model inference and Blender-based 3D generation.

Minimum Requirements:

- Processor: Intel Core i3 or equivalent (dual-core)
- RAM: 4 GB
- Storage: 250 GB HDD or SSD
- GPU: Integrated graphics (sufficient for basic Blender rendering)
- Display: Standard 13"–15.6" monitor
- Camera: Not mandatory, as floor plans are loaded from files
- Network: Basic internet connection (optional, for downloading dependencies or models)

Recommended Requirements:

- Processor: Intel Core i5/i7 or AMD equivalent (quad-core or better)
- RAM: 8 GB or higher
- Storage: 256–500 GB SSD (faster project load and export)
- GPU: Dedicated GPU (e.g., NVIDIA) for smoother 3D viewing and rendering
- Display: Full HD monitor or larger
- Network: Stable broadband connection (for package installation and model updates)

5.2 Software Requirements

The software requirements define the operating systems, programming tools, libraries, and external applications needed to build and run the system end-to-end.

- Operating System:
 - Windows 10 / 11, or
 - Linux (e.g., Ubuntu), or
 - macOS (with Blender and Python support)

- Programming Language:
 - Python 3.8+
- Backend / Scripting:
 - Python scripts for preprocessing, ONNX inference, geometry extraction, and estimation
 - Blender Python scripting for 3D model generation
- Required Python Libraries:
 - OpenCV (cv2) – image loading, resizing, padding, contour extraction
 - NumPy – numerical operations and tensor manipulation
 - onnxruntime – running the room-segmentation model (ONNX)
 - (Optional) shapely or similar geometry libraries, if used for advanced polygon operations
- External Tools / Applications:
 - Blender (latest stable version used in the project) for 3D model creation and export
 - ONNX model file for room segmentation (trained beforehand)
 - A code editor such as VS Code or PyCharm
 - Git (optional) for version control

5.3 Software Description

The software used in the 2D-to-3D Floor Plan Model Generator is primarily open-source and script-based, making the system flexible, economical, and easy to extend. Below is a brief description of the main software components and their roles.

- Python3.x:
Serves as the core development language for the preprocessing pipeline, ONNX model inference, mask-to-polygon conversion, and estimation logic. Python’s rich ecosystem allows seamless integration of computer vision libraries, ML runtimes, and file handling.
- OpenCV(cv2):
Used to load floor plan images, resize them, add padding, convert color spaces, and extract contours from the binary room mask. OpenCV provides the basic computer vision operations needed before and after ML inference.
- ONNX Runtime:

Executes the trained room-segmentation neural network stored in ONNX format. The module creates an InferenceSession, feeds the preprocessed image tensor, and receives the output mask. This makes the ML part of the pipeline portable across different platforms without retraining.

- NumPy:
Handles tensor operations, array reshaping, normalization, and numerical calculations required throughout preprocessing, inference, and estimation modules.
- Estimation Module (Python):
Implements heuristic functions that analyze walls and room polygons to estimate the number of rooms, number of wall segments, total wall length, total area, and approximate work days. It uses geometric formulas such as edge length and polygon area (shoelace formula).
- Blender & Blender Python API:
Blender is used as the 3D engine for generating the final model. A Python script running inside Blender reads the structured plan data (rooms, walls, thickness, height, scale), creates floor surfaces, extrudes walls, and exports the resulting model as GLB/GLTF or OBJ. This allows the system to transform 2D geometry into interactive 3D scenes.
- Development Tools (VS Code / PyCharm, Git):
A modern code editor is used for writing and organizing the Python and Blender scripts. Git can be used to track changes and collaborate if the project is extended by multiple developers.

These components together form a complete, open-source toolchain that supports the entire pipeline from 2D floor plan input, through ML-based room detection, to 3D model generation and basic construction estimation.

CHAPTER 6

SYSTEM DESIGN

6.1 System Architecture

The system architecture of the 2D-to-3D Floor Plan Model Generator organizes the complete workflow from raw floor plan input to final 3D model and metrics. It is divided into five logical layers: Input Acquisition, Preprocessing & ML Inference, Geometric Processing, 3D Model Generation, and Output & Visualization.

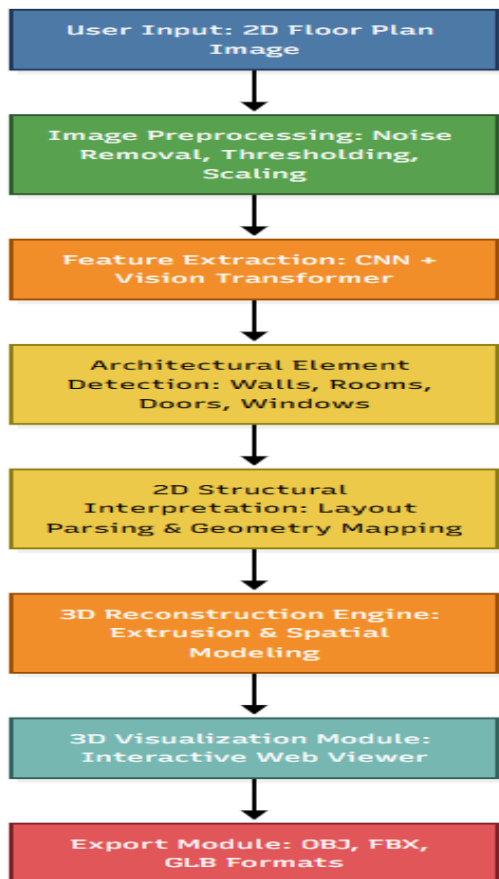


Figure 6.1 System Architecture

(You will draw a block diagram matching the five layers: Input → Preprocessing & ML → Geometry → Blender → Outputs.)

1. Input Acquisition Layer

The system accepts floor plans either as raster images (PNG/JPG) or as CAD

drawings (DXF).

- Image plans are read from disk using OpenCV.
- DXF plans are parsed using a dedicated CAD reader.

Basic validation (file type, existence) is performed at this layer.

2. Preprocessing & ML Inference Layer

For image inputs, the floor plan is resized, padded, and converted from BGR to RGB. A fixed input size (for example, 1024×1024) is used so it matches the trained model's expected input. The preprocessed image is then passed to a deep-learning room-segmentation model exported to ONNX and executed using ONNX Runtime. The model outputs a room mask, which is thresholded into a binary image indicating room regions. This layer encapsulates all ML-related operations: loading the ONNX model, running inference, and obtaining a room mask for the original floor plan.

3. Geometric Processing Layer

The binary mask is converted to vector polygons using contour extraction. Each contour is scaled back to the original coordinate system, removing padding and rescaling based on the earlier preprocessing step. These polygons represent rooms or enclosed regions.

In parallel, wall and boundary information from DXF files (if available) is incorporated into the same internal representation. A heuristic estimation module then analyzes walls and rooms to compute the number of rooms, number of wall segments, total wall length, total enclosed area, and estimated work days.

4. 3D Model Generation Layer

The processed plan data (rooms, walls, and parameters such as scale, wall thickness, and height) is passed to a Blender script. Using the Blender Python API, the script:

- Generates floor surfaces from room polygons.
- Extrudes walls based on wall paths and thickness.
- Applies height to create a 3D building shell.

The resulting mesh is optimized and prepared for export.

5. Output & Visualization Layer

The final 3D model is exported in standard formats like GLB/GLTF and OBJ. These files can be:

- Viewed in a web-based 3D viewer,
- Imported into design tools for further editing,

- Used together with the numeric estimates (area, wall length, estimated work days) for analysis or reporting.

6.2 Data Flow Diagram (DFD)

The Data Flow Diagram describes how data moves between the user, the preprocessing module, the ONNX segmentation model, the estimation module, and the Blender-based 3D generator.

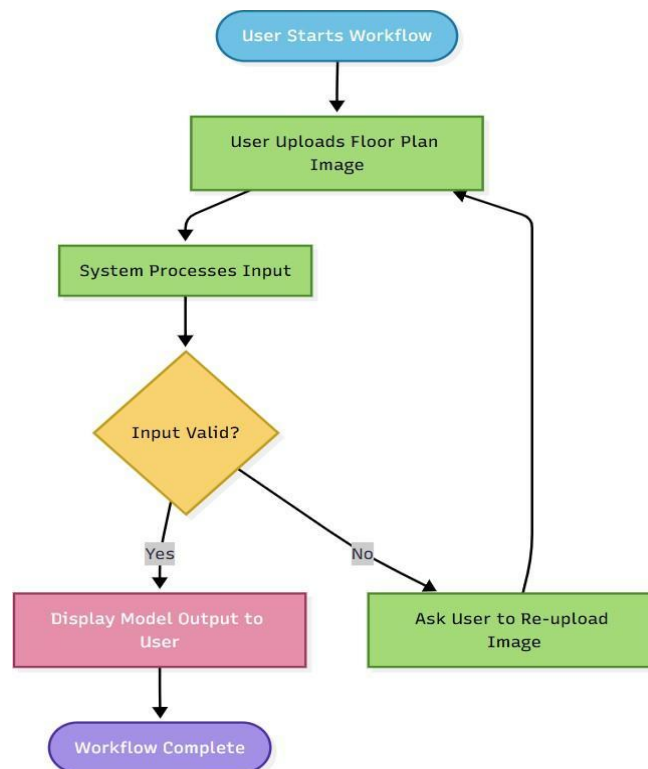


Figure 6.2 Data Flow Diagram

(You will draw a DFD: User → Input → Preprocess & ONNX → Geometry & Estimation → Blender → Outputs.)

DFD Overview

1. User → Input Module (Floor Plan File)

The user selects or provides a floor plan (image or DXF). The file path and configuration parameters (scale, wall height, wall thickness) are captured.

2. Input Module → Preprocessing & ML Inference (Image Data)

- For images: the floor plan is loaded and preprocessed (resize, pad, color

conversion).

- For DXF: geometry is read and passed as vector data.

Preprocessed image tensors are then fed into the ONNX segmentation model via ONNX Runtime.

3. ML Inference → Geometric Processing (Room Mask)

The ONNX model outputs a room mask, which is thresholded and converted to a binary mask. This mask is passed to the contour extraction function, which returns room polygons in original coordinates.

4. Geometric Processing → Estimation Module (Polygons & Walls)

Detected rooms and wall polygons are analyzed. The estimation module computes:

- Number of rooms,
- Approximate number of walls,
- Total wall length,
- Total floor area,
- Estimated work days based on area.

5. Geometric Processing → 3D Model Generator (Structured Plan Data)

The structured plan data (rooms, walls, metrics, and user parameters) is serialized and sent to the Blender script.

6. 3D Model Generator → Output Module (3D Model Files)

Blender creates the 3D mesh and exports GLB/GLTF and/or OBJ files. These are stored on disk and their locations are returned.

7. Output Module → User (3D Model + Metrics)

The user receives the exported 3D model files along with the estimated statistics (area, wall length, work days), which can be displayed in a UI or report.

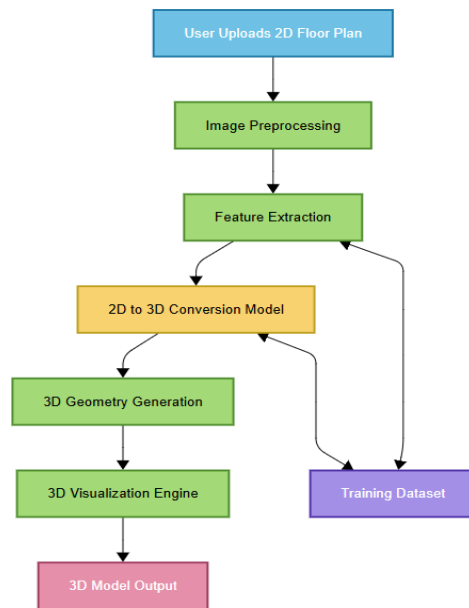


Figure 6.3 Flow Diagram

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 Module Description

The 2D-to-3D Floor Plan Model Generator is implemented as a set of modular components that work together to perform image loading, ML-based room detection, geometric processing, estimation, and 3D model creation. Each module has a clear responsibility, which simplifies testing and future extension.

7.1.1 Image Loading & Preprocessing Module

This module is responsible for reading the input floor plan image and preparing it for the ONNX segmentation model.

- Loads the image from disk using OpenCV in BGR format.
- Computes a scale factor so that the longer side fits a fixed input size (e.g., 1024 pixels).
- Resizes the image and adds padding on all sides to obtain a square input.
- Converts the padded image from BGR to RGB, normalizes pixel values to [0,1]
- [0,1], and rearranges channels to match the model's expected tensor shape (1,C,H,W)
- (1,C,H,W).
- Stores metadata such as original height/width, scale, and padding so that outputs can be mapped back to original coordinates.

This module ensures that any arbitrary floor plan image is converted into a consistent format suitable for ML inference.

7.1.2 ONNX Room Segmentation & Mask Generation Module

This module encapsulates the deep-learning part of the system.

- Loads the trained room-segmentation model from an ONNX file using ONNX Runtime (CPUExecutionProvider).
- Runs inference on the preprocessed image tensor to obtain room mask logits.
- Squeezes and thresholds the logits (e.g., at 0.5) to obtain a binary mask where 1

represents room regions.

- Handles cases where the model is unavailable or ONNX Runtime is not installed by returning empty results gracefully.

The output of this module is a room mask aligned with the padded, resized image.

7.1.3 Mask-to-Polygon Conversion Module

This module converts the binary room mask into vector polygons that can be used for geometry and 3D generation.

- Uses OpenCV contour extraction to find external contours in the mask.
- Filters out very small contours below a minimum area threshold to remove noise.
- For each contour point, reverses the padding and scale operations to map coordinates back to the original image space.
- Outputs a list of polygons, each represented as an ordered list of
 - [x,y]
 - [x,y] points.

This step transforms pixel-based segmentation into clean, scalable geometric primitives.

7.1.4 Plan Estimation Module

This module computes high-level estimates from the parsed plan (from polygons and optional walls).

- Receives a plan dictionary containing walls and rooms.
- Uses polygon edge lengths to approximate total wall length.
- Applies the shoelace formula to compute total enclosed area for wall and room polygons.
- Estimates number of rooms, number of wall segments, and approximate work days based on area (e.g., assuming a fixed square-meter-per-day productivity).

When only image dimensions are available, a simplified estimator uses image size and scale to approximate area, room count, wall count, and work days.

7.1.5 3D Model Generation Module (Blender Script)

This module runs inside Blender and is responsible for creating the final 3D model.

- Reads the structured plan data (rooms, walls, thickness, height, scale).
- Creates 2D floor surfaces from room polygons.
- Extrudes walls along wall paths with specified thickness and height.
- Applies basic materials or colors for visual clarity.
- Exports the resulting 3D model as GLB/GLTF or OBJ for external viewing.

This module connects the analytical part of the pipeline to a tangible, visual 3D output.

7.1.6 Orchestration & Error Handling Module

A lightweight orchestration layer ties all modules together.

- Validates inputs and configuration (paths, model availability, scale values).
- Calls the image preprocessing, ONNX inference, mask-to-polygon, and estimation modules in order.
- Prepares the final data structures for Blender and triggers the Blender script.
- Handles missing dependencies (OpenCV, ONNX Runtime, model file) by returning safe default outputs instead of crashing.

This design keeps the system robust and easier to debug.

CHAPTER 8

SYSTEM TESTING AND MAINTENANCE

8.1 UNIT TESTING

Unit testing was performed on each core module of the 2D-to-3D Floor Plan Model Generator to verify that individual functions behaved correctly in isolation.

1. Image Loading & Preprocessing Module

- Verified that valid image paths were loaded correctly using OpenCV.
- Checked that resizing and padding produced the expected 1024×1024 tensor while preserving aspect ratio.
- Confirmed that metadata (scale, padding, original size) was correct for different input dimensions.

2. ONNX Room Segmentation Module

- Tested model loading with valid and invalid ONNX file paths.
- Verified that inference produced a mask of expected shape and data type.
- Confirmed that thresholding of logits generated a binary mask without crashes or shape mismatches.

3. Mask-to-Polygon Conversion Module

- Checked contour detection on synthetic masks with simple rectangular rooms.
- Verified that very small contours were filtered out as noise.
- Confirmed that coordinates were correctly mapped back from padded space to original image space.

4. Plan Estimation Module

- Tested polygon area computation using known shapes (rectangles, triangles) to validate the shoelace formula.
- Verified that wall length and room counts were computed consistently for sample plan data.
- Checked the fallback estimator (from image size only) for reasonable approximate values.

5. Blender 3D Generation Script

- Verified that the script could import plan data, create floor meshes, extrude

walls, and export GLB/OBJ files.

- Ensured that invalid or empty input data did not cause Blender to crash but produced appropriate error messages.

6. Orchestration & Error Handling

- Tested missing dependencies (no OpenCV, no ONNX Runtime, missing model file) to ensure the system returned safe default outputs.
- Checked that incorrect configuration parameters (e.g., negative scale) were detected and handled gracefully.

8.2 INTEGRATION TESTING

After unit tests, integration testing verified that modules interacted correctly when combined into a full pipeline.

- Confirmed that preprocessed image tensors from the preprocessing module were compatible with the ONNX segmentation module.
- Verified that the room mask produced by the ONNX model could be passed directly to the mask-to-polygon module.
- Checked that the polygons generated were accepted by the estimation module and by the Blender script without format issues.
- Ensured that file paths and configuration values flowed correctly from the orchestration layer to Blender and back.

Integration testing validated that the data flow between modules (image → tensor → mask → polygons → 3D model) was stable and consistent.

8.3 FUNCTIONAL TESTING

Functional testing evaluated whether the system behaved correctly from an end-user point of view based on the specified requirements.

- The user could select a 2D floor plan image or DXF file and provide parameters such as scale, wall height, and wall thickness.
- The system successfully generated room polygons from the ONNX segmentation model and produced a 3D model in Blender.

- Exported GLB/OBJ files opened correctly in external 3D viewers.
- Estimated values (number of rooms, total wall length, total area, estimated work days) were displayed and matched expectations for test plans.
- Error messages were shown when invalid files or missing models were used.

These tests confirmed that the main features of the 2D-to-3D conversion process worked as intended.

8.4 SYSTEM TESTING

System testing was performed on the fully integrated application to ensure that it met overall performance and reliability expectations.

- Tested the complete workflow: Input → Preprocessing & ONNX → Polygons & Estimation → Blender → 3D Output.
- Evaluated performance with floor plans of different sizes and complexities.
- Measured the approximate time taken from providing the input file to obtaining the final 3D model and estimates.
- Verified stability under repeated runs with different parameters and multiple test floor plans.
- Checked that the system behaved consistently across different operating systems where possible.

System testing ensured that the application functioned correctly as a whole, not just at the module level.

8.5 WHITE BOX TESTING

White box testing focused on the internal logic and code paths.

- Reviewed preprocessing logic to ensure that scale and padding calculations were correct for all branches.
- Examined the thresholding and shape handling in the ONNX inference module to avoid indexing errors.
- Checked loops in the estimation module for correct handling of edge cases, such as empty polygon lists.

- Verified that exceptions related to file I/O and missing dependencies were caught and processed correctly.

White box testing confirmed that internal implementations matched the design and handled edge cases safely.

8.6 BLACK BOX TESTING

Black box testing validated system behavior based only on inputs and outputs, without inspecting internal code.

- Tested a variety of floor plans with different layouts, room shapes, and image qualities.
- Verified that the system produced a visually correct 3D model for each input and that room detection was reasonable.
- Checked that changing parameters such as scale or wall height affected the output as expected.
- Confirmed that invalid inputs (wrong file type, corrupted images) resulted in clear error messages rather than crashes.

This confirmed that the system delivered reliable and user-friendly behavior under realistic usage scenarios.

8.7 MAINTENANCE

Maintenance activities ensure that the system remains usable and up-to-date over time.

- **Updating Dependencies:** Periodically verify compatibility with newer versions of Python, OpenCV, ONNX Runtime, and Blender.
- **Model Updates:** Replace or retrain the ONNX room-segmentation model to improve accuracy or support new types of floor plans.
- **Bug Fixes:** Monitor user feedback, reproduce reported issues, and patch defects in preprocessing, estimation, or Blender scripts.
- **Performance Tuning:** Optimize preprocessing and inference steps if larger floor plans or higher resolution inputs are used.
- **Documentation:** Keep user and developer documentation up-to-date so that future students or developers can extend the system easily.

CHAPTER 9

9. FUTURE ENHANCEMENT AND CONCLUSION

The 2D-to-3D Floor Plan Model Generator already provides a practical solution for automatically converting 2D floor plans into metric 3D models using an ONNX-based room-segmentation model and Blender automation. However, there are several directions in which the system can be improved.

Future enhancements could include integrating more advanced deep learning models that jointly detect walls, doors, and windows, instead of only room regions. Multi-class semantic segmentation or instance segmentation networks could provide richer labels, enabling more detailed 3D reconstructions with doors and windows embedded directly into walls. Support for multiple floors and stair detection could be added to generate complete multi-storey building models. The system could also incorporate more accurate quantity estimation by including material types and construction standards.

Another improvement would be to develop a graphical user interface where users can visually inspect the detected rooms, make corrections if needed, and then regenerate the 3D model. A web-based or desktop UI could also display real-time statistics (area, wall length, estimated work days) and allow exporting reports. Cloud-based deployment or containerization would make it easier to run the system on shared infrastructure and integrate it into larger design or BIM workflows.

Overall, this project demonstrates how modern computer vision, deep learning segmentation, and 3D graphics scripting can automate a traditionally manual and time-consuming task. By converting 2D floor plans into accurate 3D models and providing basic construction-related estimates, the system reduces effort for architects, engineers, and students, while promoting faster design exploration and better visualization. With further work on model accuracy, input variety, user interfaces, and multi-storey support, the 2D-to-3D Floor Plan Model Generator has strong potential to evolve into a powerful and accessible tool for architectural visualization and education.

CHAPTER 10

APPENDIX

A.1 SOURCE CODE

```
#!/usr/bin/env python3
"""
2D to 3D Building Blueprint Converter - Main Entry Point
"""

import argparse
import json
import logging
import os
import subprocess
import sys
from pathlib import Path

import parser
import map_downloader

# Custom JSON encoder to handle numpy types
class NumpyEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, (np.integer, np.floating)):
            return float(obj)
        elif isinstance(obj, np.ndarray):
            return obj.tolist()
        return super().default(obj)

# Try to import numpy for encoder
try:
```

```

import numpy as np
except ImportError:
    np = None

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

def check_blender_available():
    """Check if Blender is available in PATH."""
    try:
        result = subprocess.run(
            ['blender', '--version'],
            capture_output=True,
            text=True,
            timeout=10
        )
        if result.returncode == 0:
            logger.info(f"Blender found: {result.stdout.strip().split()[1]}")
            return True
    except (subprocess.TimeoutExpired, FileNotFoundError, subprocess.SubprocessError):
        pass

    logger.error("Blender not found. Please install Blender 3.5+ and ensure it's in PATH.")
    logger.error("Alternatively, set BLENDER_PATH environment variable.")
    return False

def call_blender(plan_json_path, height, output_dir, wall_thickness=0.2, stair_width=1.5,
furniture_enabled=True, map_path=None, door_height=None, door_width=1.0):

```

```
"""
```

Call Blender in background mode to generate 3D model.

Args:

plan_json_path: Path to plan.json file

height: Building height in meters

output_dir: Output directory for generated files

wall_thickness: Wall thickness in meters

stair_width: Stair width in meters

furniture_enabled: Whether to generate furniture

map_path: Path to map file (mbtiles)

door_height: Door height in meters (defaults to height * 0.85)

door_width: Door width in meters

```
"""
```

```
blender_path = os.environ.get('BLENDER_PATH', 'blender')
```

```
blender_script = Path(__file__).parent / 'blender_generator.py'
```

```
# Build Blender command
```

```
cmd = [  
    blender_path,  
    '--background',  
    '--python', str(blender_script),  
    '--',  
    '--plan', plan_json_path,  
    '--height', str(height),  
    '--out', output_dir,  
    '--wall-thickness', str(wall_thickness),  
    '--stair-width', str(stair_width),  
    '--door-width', str(door_width)  
]
```

```
if not furniture_enabled:
```

```
    cmd.append('--no-furniture')
```

```

if map_path:
    cmd.extend(['--map', map_path])

if door_height is not None:
    cmd.extend(['--door-height', str(door_height)])

logger.info("Calling Blender to generate 3D model...")
logger.debug(f"Command: {' '.join(cmd)}")

try:
    result = subprocess.run(
        cmd,
        capture_output=True,
        text=True,
        timeout=300 # 5 minute timeout
    )

    if result.returncode != 0:
        logger.error(f"Blender execution failed with code {result.returncode}")
        logger.error("Blender stdout:")
        logger.error(result.stdout)
        logger.error("Blender stderr:")
        logger.error(result.stderr)
        return False

    logger.info("Blender execution completed successfully")
    if result.stdout:
        logger.debug("Blender output:")
        logger.debug(result.stdout)
    return True

except subprocess.TimeoutExpired:
    logger.error("Blender execution timed out (>5 minutes)")
    return False

```

```

except Exception as e:
    logger.error(f"Error calling Blender: {e}")
    return False


def parse_optional_json(flag_value):
    """Parse JSON from string or file path."""
    if not flag_value:
        return None

    # Try to parse as JSON string
    try:
        return json.loads(flag_value)
    except json.JSONDecodeError:
        pass

    # Try to read from file
    if os.path.exists(flag_value):
        try:
            with open(flag_value, 'r') as f:
                return json.load(f)
        except Exception as e:
            logger.error(f"Failed to read JSON from file {flag_value}: {e}")

    logger.error(f"Invalid JSON: {flag_value}")
    return None


def serve_viewer(output_dir):
    """Launch a simple HTTP server for the viewer."""
    import http.server
    import socketserver
    import webbrowser
    from pathlib import Path

```

```
PORT = 8000
```

```
# Create handler that serves files relative to project root
```

```
class CustomHandler(http.server.SimpleHTTPRequestHandler):
```

```
    def __init__(self, *args, **kwargs):
```

```
        super().__init__(*args, directory=str(Path(__file__).parent), **kwargs)
```

```
try:
```

```
    with socketserver.TCPServer(("", PORT), CustomHandler) as httpd:
```

```
        url = f"http://localhost:{PORT}/viewer/index.html"
```

```
        logger.info(f"Serving viewer at {url}")
```

```
        webbrowser.open(url)
```

```
        logger.info("Press Ctrl+C to stop the server")
```

```
        httpd.serve_forever()
```

```
except KeyboardInterrupt:
```

```
    logger.info("Server stopped")
```

```
except OSError as e:
```

```
    logger.error(f"Failed to start server on port {PORT}: {e}")
```

```
def main():
```

```
    """Main entry point."""
```

```
    parser_args = argparse.ArgumentParser(
```

```
        description='Convert 2D building blueprints to 3D models',
```

```
        formatter_class=argparse.RawDescriptionHelpFormatter
```

```
    )
```

```
    parser_args.add_argument(
```

```
        '--input',
```

```
        required=False,
```

```
        help='Path to blueprint file (PNG/JPG/DXF)'
```

```
    )
```

```

parser_args.add_argument(
    '--output',
    default='output',
    help='Output directory (default: output/)'
)
parser_args.add_argument(
    '--height',
    type=float,
    default=3.0,
    help='Building height in meters (default: 3.0)'
)
parser_args.add_argument(
    '--scale',
    type=float,
    default=100.0,
    help='Pixels per meter for image inputs (default: 100)'
)
parser_args.add_argument(
    '--stairs',
    help='JSON string or file path for stair locations'
)
parser_args.add_argument(
    '--entries',
    help='JSON string or file path for door coordinates'
)
parser_args.add_argument(
    '--serve-viewer',
    action='store_true',
    help='Launch a web server to preview the model'
)
parser_args.add_argument(
    '--download-maps',
    action='store_true',
    help='Download OSM tiles for context (optional)'
)

```

```

)
parser_args.add_argument(
    '--wall-thickness',
    type=float,
    default=0.2,
    help='Wall thickness in meters (default: 0.2)'
)
parser_args.add_argument(
    '--stair-width',
    type=float,
    default=1.2,
    help='Stair width in meters (default: 1.2)'
)
parser_args.add_argument(
    '--no-furniture',
    action='store_true',
    help='Disable furniture generation'
)
parser_args.add_argument(
    '--map',
    type=str,
    default=None,
    help='Path to map file (mbtiles) for overlay'
)
parser_args.add_argument(
    '--ui',
    action='store_true',
    help='Launch GUI mode'
)
parser_args.add_argument(
    '--ml',
    action='store_true',
    help='Enable ML-based enhancement (rooms/labels) for image blueprints'
)

```



```

parser_args.add_argument(
    '--ml-model',
    type=str,
    default=None,
    help='Path to ONNX model for ML enhancement'
)
parser_args.add_argument(
    '--ml-task',
    type=str,
    default='rooms',
    help='ML task (e.g., rooms)'
)
parser_args.add_argument(
    '--no-blender',
    action='store_true',
    help='Skip calling Blender and continue to save plan.json'
)

args = parser_args.parse_args()

# Handle UI mode
if args.ui:
    try:
        import ui
        ui.launch_ui()
        return 0
    except ImportError:
        logger.error("UI module not found. Run without --ui flag for CLI mode.")
        return 1

# For CLI mode, input is required
if not args.input:
    logger.error("Error: --input is required for CLI mode")
    return 1

```

```

# Verify input file exists
if not os.path.exists(args.input):
    logger.error(f"Input file not found: {args.input}")
    return 1

# Create output directory
output_dir = Path(args.output)
output_dir.mkdir(parents=True, exist_ok=True)

logger.info("=" * 60)
logger.info("2D to 3D Building Blueprint Converter")
logger.info("=" * 60)
logger.info(f"Input: {args.input}")
logger.info(f"Output: {args.output}")
logger.info(f"Height: {args.height}m")

# Parse optional JSON parameters
stairs_data = parse_optional_json(args.stairs) if args.stairs else None
entries_data = parse_optional_json(args.entries) if args.entries else None

# Parse blueprint
logger.info("Parsing blueprint...")
try:
    plan_data = parser.parse_blueprint(
        args.input,
        pixels_per_meter=args.scale,
        stairs_data=stairs_data,
        entries_data=entries_data
    )

    if not plan_data:
        logger.error("Failed to parse blueprint")
        return 1

```

```

logger.info(f"Detected {len(plan_data['walls'])} walls")
logger.info(f"Detected {len(plan_data['doors'])} doors")
logger.info(f"Detected {len(plan_data['stairs'])} stair regions")
logger.info(f"Detected {len(plan_data.get('furniture', []))} furniture regions")

except Exception as e:
    logger.error(f"Error parsing blueprint: {e}", exc_info=True)
    return 1

# Optional ML enhancement for image inputs
try:
    file_ext = os.path.splitext(args.input)[1].lower()
    if args.ml and file_ext in ['.png', '.jpg', '.jpeg', '.bmp']:
        import cv2 # local import to avoid hard dependency if ML disabled
        from ml.inference import detect_rooms_and_objects
        from ml.postprocess import merge_ml_into_plan

        logger.info("Running ML enhancement...")
        img = cv2.imread(args.input, cv2.IMREAD_COLOR)
        if img is None:
            logger.warning("Cannot load image for ML; skipping ML enhancement.")
        else:
            h, w = img.shape[:2]
            ml_out = detect_rooms_and_objects(args.input, model_path=args.ml_model,
task=args.ml_task)
            plan_data = merge_ml_into_plan(plan_data, ml_out, pixels_per_meter=args.scale,
img_width=w, img_height=h)

            logger.info(f"ML enhancement added {len(plan_data.get('rooms', []))} rooms")
except Exception as e:
    logger.warning(f"ML enhancement failed: {e}")

# Add metadata
plan_data['metadata'] = {

```

```

'scale': args.scale,
'height': args.height,
'input_file': args.input
}

# Save plan.json
plan_json_path = output_dir / 'plan.json'
try:
    with open(plan_json_path, 'w') as f:
        encoder = NumpyEncoder if np else json.JSONEncoder
        json.dump(plan_data, f, indent=2, cls=encoder)
        logger.info(f"Saved plan data to {plan_json_path}")
except Exception as e:
    logger.error(f"Failed to save plan.json: {e}")
    return 1

# Optionally download maps
if args.download_maps:
    logger.info("Downloading OSM tiles...")
    try:
        map_downloader.download_tiles(plan_data['bounds'], output_dir)
    except Exception as e:
        logger.warning(f"Map download failed: {e}")

# Check Blender availability unless user chose to skip it
if not args.no_blender:
    if not check_blender_available():
        logger.error("Cannot proceed without Blender. Install Blender 3.5+ and ensure it's in
PATH.")
        return 1
    else:
        logger.info("Skipping Blender invocation because --no-blender was specified")

# Generate 3D model with Blender (skip if requested)

```

```

if not args.no_blender:
    if not call_blender(
        str(plan_json_path),
        args.height,
        str(output_dir),
        wall_thickness=args.wall_thickness,
        stair_width=args.stair_width,
        furniture_enabled=not args.no_furniture,
        map_path=args.map,
        door_height=args.height * 0.85 if args.height else None,
        door_width=1.0
    ):
        logger.error("Failed to generate 3D model")
        return 1
    else:
        logger.info("Blender generation skipped; plan.json saved in output directory")

logger.info("=" * 60)
logger.info("Conversion complete!")
logger.info(f"Output files in: {output_dir}")
logger.info(f"- plan.json: Parsed blueprint data")
logger.info(f"- model.glb: 3D model (web-ready)")
logger.info(f"- model.obj: 3D model (standard format)")
logger.info("=" * 60)

# Generate report (plan.obj, blueprint.svg, report.html)
try:
    import subprocess
    report_script = Path(__file__).parent / 'generate_report.py'
    result = subprocess.run(
        ['python', str(report_script), '--plan', str(plan_json_path), '--out', str(output_dir)],
        capture_output=True,
        text=True,
        timeout=30

```

```

    )
    if result.returncode == 0:
        logger.info("Report files generated successfully")
    else:
        logger.warning(f"Report generation failed: {result.stderr}")
except Exception as e:
    logger.warning(f"Could not generate report: {e}")

# Serve viewer if requested
if args.serve_viewer:
    serve_viewer(output_dir)

return 0

if __name__ == '__main__':
    sys.exit(main())

```

ui.py

```

#!/usr/bin/env python3
"""
Simple Tkinter GUI for 2D to 3D Building Blueprint Converter
"""

import tkinter as tk
from tkinter import filedialog, messagebox, ttk
import threading
import sys
import os
from pathlib import Path
from PIL import Image, ImageTk, ImageDraw

```

```

import subprocess
import json

# Import main functionality
import main as main_module

class BlueprintConverterGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("2D to 3D Building Blueprint Converter")
        self.root.geometry("1400x900")

        # Make fullscreen
        self.root.state('zoomed') # Windows fullscreen

        # Variables
        self.input_file = tk.StringVar()
        self.output_dir = tk.StringVar(value="output")
        self.height = tk.DoubleVar(value=3.5)
        self.scale = tk.DoubleVar(value=100.0)
        self.wall_thickness = tk.DoubleVar(value=0.2)
        self.stair_width = tk.DoubleVar(value=1.2)
        self.show_furniture = tk.BooleanVar(value=True)
        self.show_map = tk.BooleanVar(value=False)
        self.map_file = tk.StringVar()
        self.skip_blender = tk.BooleanVar(value=False)

        self.create_widgets()

    def create_widgets(self):
        # Use a horizontal paned layout: left controls, right preview
        paned = ttk.Panedwindow(self.root, orient=tk.HORIZONTAL)
        paned.pack(fill=tk.BOTH, expand=True)

```

```

controls = ttk.Frame(paned, width=420, padding=12)
preview = ttk.Frame(paned, padding=12)
paned.add(controls, weight=1)
paned.add(preview, weight=3)

# Controls (left)
row = 0
title = ttk.Label(controls, text="2D → 3D Converter", font=("Segoe UI", 18, "bold"))
title.grid(row=row, column=0, columnspan=3, pady=(0, 12), sticky=tk.W)
row += 1

ttk.Label(controls, text="Blueprint File:").grid(row=row, column=0, sticky=tk.W,
pady=4)
ttk.Entry(controls, textvariable=self.input_file, width=30).grid(row=row, column=1,
sticky=(tk.W, tk.E))
ttk.Button(controls, text="Browse", command=self.browse_input).grid(row=row,
column=2, padx=6)
row += 1

ttk.Label(controls, text="Output Directory:").grid(row=row, column=0, sticky=tk.W,
pady=4)
ttk.Entry(controls, textvariable=self.output_dir, width=24).grid(row=row, column=1,
sticky=(tk.W, tk.E))
ttk.Button(controls, text="Browse", command=self.browse_output).grid(row=row,
column=2, padx=6)
row += 1

self.output_path_label = ttk.Label(controls, text=f"📁 Path:
{Path(self.output_dir.get()).resolve()}", foreground="#666")
self.output_path_label.grid(row=row, column=0, columnspan=3, sticky=tk.W,
pady=(2,8))
self.output_dir.trace('w', self.update_output_path_label)

```



```

row += 1

# Parameters
ttk.Separator(controls, orient='horizontal').grid(row=row, column=0, columnspan=3,
sticky=(tk.W, tk.E), pady=8)
row += 1

ttk.Label(controls, text="Height (m):").grid(row=row, column=0, sticky=tk.W)
ttk.Spinbox(controls, from_=1.0, to=50.0, textvariable=self.height,
width=10).grid(row=row, column=1, sticky=tk.W)
row += 1

ttk.Label(controls, text="Pixels / Meter:").grid(row=row, column=0, sticky=tk.W)
ttk.Spinbox(controls, from_=10, to=800, textvariable=self.scale,
width=10).grid(row=row, column=1, sticky=tk.W)
row += 1

ttk.Label(controls, text="Wall thickness (m):").grid(row=row, column=0, sticky=tk.W)
ttk.Spinbox(controls, from_=0.05, to=2.0, increment=0.01,
textvariable=self.wall_thickness, width=10).grid(row=row, column=1, sticky=tk.W)
row += 1

ttk.Label(controls, text="Stair width (m):").grid(row=row, column=0, sticky=tk.W)
ttk.Spinbox(controls, from_=0.5, to=3.0, increment=0.1, textvariable=self.stair_width,
width=10).grid(row=row, column=1, sticky=tk.W)
row += 1

# Options
ttk.Separator(controls, orient='horizontal').grid(row=row, column=0, columnspan=3,
sticky=(tk.W, tk.E), pady=8)
row += 1

ttk.Checkbutton(controls, text="Generate Furniture",
variable=self.show_furniture).grid(row=row, column=0, columnspan=2, sticky=tk.W)
row += 1

```

```

        ttk.Checkbutton(controls, text="Skip Blender (save plan only)",
variable=self.skip_blender).grid(row=row, column=0, columnspan=2, sticky=tk.W)
        row += 1
        ttk.Checkbutton(controls, text="Show Map Overlay", variable=self.show_map,
command=self.toggle_map_input).grid(row=row, column=0, columnspan=2, sticky=tk.W)
        row += 1

# Map input
self.map_frame = ttk.Frame(controls)
ttk.Label(self.map_frame, text="Map File:").grid(row=0, column=0, sticky=tk.W)
ttk.Entry(self.map_frame, textvariable=self.map_file, width=20).grid(row=0, column=1,
sticky=(tk.W, tk.E))
        ttk.Button(self.map_frame, text="Browse", command=self.browse_map).grid(row=0,
column=2)
        self.map_frame.grid(row=row, column=0, columnspan=3, sticky=(tk.W, tk.E), pady=4)
        self.map_frame.grid_remove()
        row += 1

# Action buttons
        ttk.Separator(controls, orient='horizontal').grid(row=row, column=0, columnspan=3,
sticky=(tk.W, tk.E), pady=8)
        row += 1

        btn_frame = ttk.Frame(controls)
        btn_frame.grid(row=row, column=0, columnspan=3, pady=6)
        self.convert_button = ttk.Button(btn_frame, text="Convert",
command=self.start_conversion)
        self.convert_button.pack(side=tk.LEFT, padx=6)
        self.report_button = ttk.Button(btn_frame, text="Open Report",
command=self.open_report, state='disabled')
        self.report_button.pack(side=tk.LEFT, padx=6)
        ttk.Button(btn_frame, text="Generate Samples",
command=self.generate_samples).pack(side=tk.LEFT, padx=6)
        ttk.Button(btn_frame, text="Exit", command=self.root.quit).pack(side=tk.LEFT,

```

```

padx=6)

    row += 1

# Status & progress
self.progress = ttk.Progressbar(controls, mode='indeterminate')
self.progress.grid(row=row, column=0, columnspan=3, sticky=(tk.W, tk.E), pady=8)
row += 1
self.status_label = ttk.Label(controls, text="Ready", foreground="green")
self.status_label.grid(row=row, column=0, columnspan=3, sticky=tk.W)

# Preview (right)
preview_title = ttk.Label(preview, text="Preview & Sketch", font=("Segoe UI", 14,
"bold"))
preview_title.pack(anchor=tk.W)
self.canvas = tk.Canvas(preview, bg="#ffffff")
self.canvas.pack(fill=tk.BOTH, expand=True, padx=8, pady=8)
self.preview_image = None

# Initial preview update
self.root.after(200, self.update_preview)

def browse_input(self):
    filename = filedialog.askopenfilename(
        title="Select Blueprint File",
        filetypes=[
            ("Image files", "*.png *.jpg *.jpeg *.bmp"),
            ("DXF files", "*.dxf"),
            ("All files", "*.*")
        ]
    )
    if filename:
        self.input_file.set(filename)

def browse_output(self):

```

```

    dirname = filedialog.askdirectory(title="Select Output Directory")
    if dirname:
        self.output_dir.set(dirname)

def browse_map(self):
    filename = filedialog.askopenfilename(
        title="Select Map File",
        filetypes=[("MBTiles", "*.mbtiles"), ("All files", "*.*")]
    )
    if filename:
        self.map_file.set(filename)

def toggle_map_input(self):
    if self.show_map.get():
        self.map_frame.grid()
    else:
        self.map_frame.grid_remove()

def start_conversion(self):
    if not self.input_file.get():
        messagebox.showerror("Error", "Please select a blueprint file")
        return
    # Disable button and start progress
    self.convert_button.config(state='disabled')
    self.progress.start()
    self.status_label.config(text="Processing...", foreground="blue")

    # Run conversion in separate thread
    thread = threading.Thread(target=self.run_conversion)
    thread.daemon = True
    thread.start()

def run_conversion(self):
    try:
        # Build arguments

```

```

sys.argv = [
    'main.py',
    '--input', self.input_file.get(),
    '--output', self.output_dir.get(),
    '--height', str(self.height.get()),
    '--scale', str(self.scale.get()),
    '--wall-thickness', str(self.wall_thickness.get()),
    '--stair-width', str(self.stair_width.get())
]

if not self.show_furniture.get():
    sys.argv.append('--no-furniture')

if self.skip_blender.get():
    sys.argv.append('--no-blender')

if self.show_map.get() and self.map_file.get():
    sys.argv.extend(['--map', self.map_file.get()])

# Run main conversion
result = main_module.main()

# Update UI in main thread
self.root.after(0, self.conversion_complete, result == 0)

except Exception as e:
    self.root.after(0, self.conversion_error, str(e))

def conversion_complete(self, success):
    self.progress.stop()
    self.convert_button.config(state='normal')

if success:
    self.status_label.config(text="Conversion complete!", foreground="green")

```

```

self.report_button.config(state='normal')
# Refresh the preview to show the newly generated plan
try:
    self.root.after(100, self.update_preview)
except Exception:
    pass
msg = f"3D model generated in:\n{self.output_dir.get()}"
if self.skip_blender.get():
    msg += "\n\nClick 'Open Report' to view the blueprint."
messagebox.showinfo("Success", msg)
else:
    self.status_label.config(text="Conversion failed", foreground="red")
    self.report_button.config(state='disabled')
    messagebox.showerror("Error", "Conversion failed. Check console for details.")

def conversion_error(self, error_msg):
    self.progress.stop()
    self.convert_button.config(state='normal')
    self.report_button.config(state='disabled')
    self.status_label.config(text="Error occurred", foreground="red")
    messagebox.showerror("Error", f"An error occurred:\n{error_msg}")

def open_report(self):
    import webbrowser
    report_path = Path(self.output_dir.get()) / 'report.html'
    if report_path.exists():
        webbrowser.open(f'file://{report_path.resolve()}')
    else:
        messagebox.showerror("Error", f"Report not found at:\n{report_path}")

def update_output_path_label(self, *args):
    """Update the output path label when directory changes."""
    output_dir = Path(self.output_dir.get())
    abs_path = output_dir.resolve()

```

```

self.output_path_label.config(text=f"📁 Path: {abs_path}")

def generate_samples(self):
    """Run the sample blueprint generator to create multiple sample inputs."""
    try:
        script = Path(__file__).parent / 'generate_sample_blueprints.py'
        result = subprocess.run(['py', str(script)], capture_output=True, text=True,
                                timeout=60)
        if result.returncode == 0:
            messagebox.showinfo('Samples Created', 'Sample blueprints generated in\nsample_input/')
        else:
            messagebox.showwarning('Generate Samples', f'Generator returned non-zero:\n{result.stderr}')
    except Exception as e:
        messagebox.showerror('Error', f'Failed to generate samples:\n{e}')

def update_preview(self):
    """Update the right-hand preview canvas with the input image or an interpreted sketch from plan.json."""
    try:
        out_dir = Path(self.output_dir.get())
        plan_path = out_dir / 'plan.json'
        if plan_path.exists():
            with open(plan_path, 'r') as f:
                plan = json.load(f)
            # Render a sketch from plan polygons
            # Determine bounds
            polys = plan.get('walls', []) or plan.get('rooms', [])
            if not polys:
                # show input image
                self._show_input_image()
            return
    
```

```

# Compute bounds
xs = [p[0] for poly in polys for p in poly]
ys = [p[1] for poly in polys for p in poly]
minx, maxx = min(xs), max(xs)
miny, maxy = min(ys), max(ys)
width = int((maxx - minx) * self.scale.get()) + 40
height = int((maxy - miny) * self.scale.get()) + 40
img = Image.new('RGBA', (max(320, width), max(240, height)), (250, 250, 250,
255))

draw = ImageDraw.Draw(img)
# draw floor
draw.rectangle([0,0,img.width,img.height], fill=(245,245,245))
def to_px(pt):
    x, y = pt
    px = int((x - minx) * self.scale.get()) + 20
    py = int((maxy - y) * self.scale.get()) + 20
    return (px, py)
# Draw polygons with pseudo-3D effect (offset shadow)
for poly in polys:
    pts = [to_px(p) for p in poly]
    # offset for height
    offset = int(self.height.get() * 4)
    shadow = [(x+offset, y-offset) for (x,y) in pts]
    draw.polygon(shadow, fill=(200,200,200,180))
    draw.polygon(pts, fill=(220,240,255), outline=(30,80,120))

# Convert to Tk image and display
self.preview_image = ImageTk.PhotoImage(img)
self.canvas.delete('all')
self.canvas.create_image(0, 0, anchor='nw', image=self.preview_image)
self.canvas.config(scrollregion=self.canvas.bbox(tk.ALL))
else:
    self._show_input_image()
except Exception as e:

```



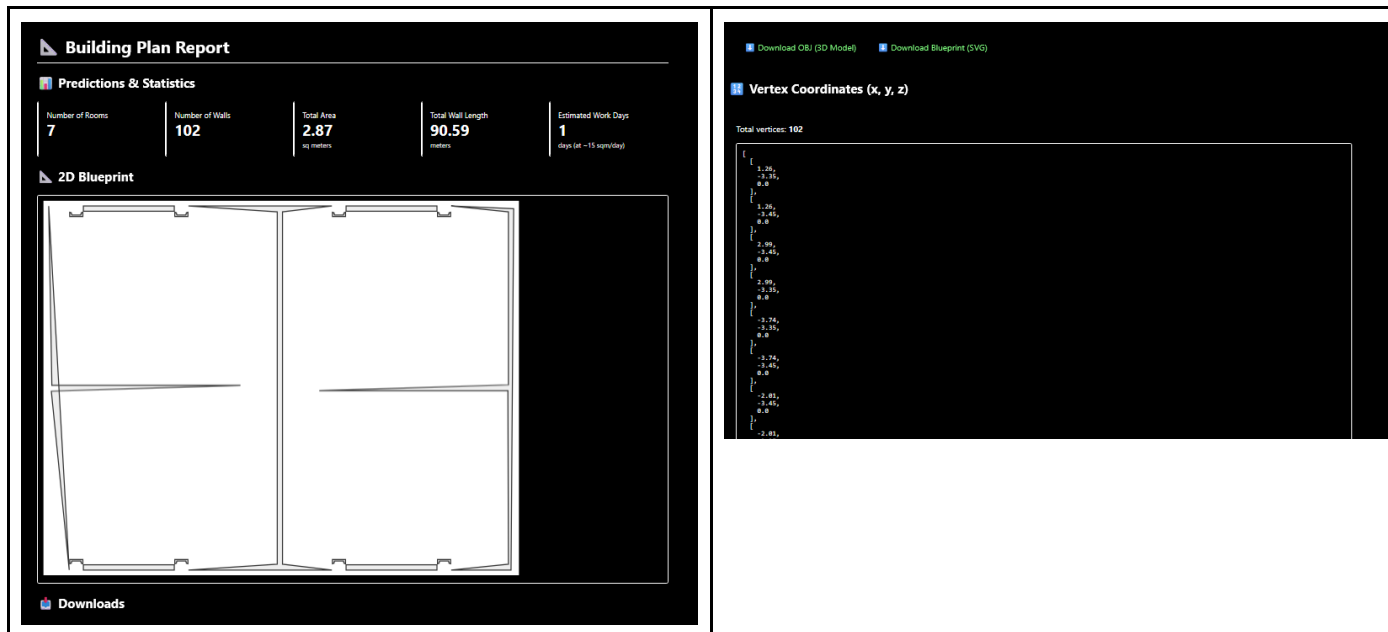
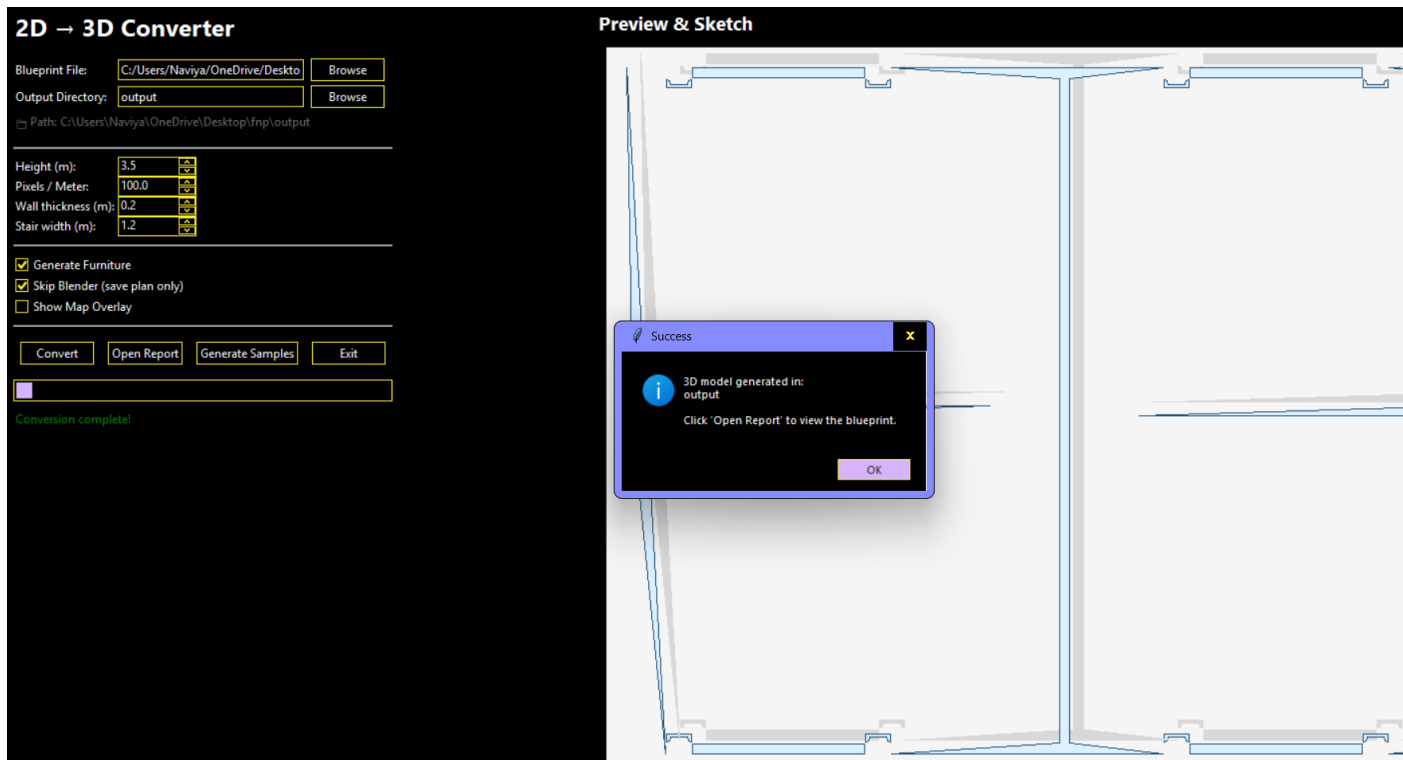
```

        print('Preview error:', e)
def _show_input_image(self):
    try:
        inp = self.input_file.get()
        if not inp:
            return
        img = Image.open(inp)
        # fit to canvas
        cw = max(400, self.canvas.winfo_width() or 600)
        ch = max(300, self.canvas.winfo_height() or 400)
        img.thumbnail((cw, ch), Image.ANTIALIAS)
        self.preview_image = ImageTk.PhotoImage(img)
        self.canvas.delete('all')
        self.canvas.create_image(10, 10, anchor='nw', image=self.preview_image)
    except Exception:
        pass
def launch_ui():
    """Launch the GUI application."""
    root = tk.Tk()
    app = BlueprintConverterGUI(root)
    root.mainloop()

if __name__ == '__main__':
    launch_ui()

```

A.2 SCREENSHOT



Building Plan Report

Predictions & Statistics

Number of Rooms

1

Number of Walls

4

Total Area

85.68

m² meters

Total Wall Length

37.88

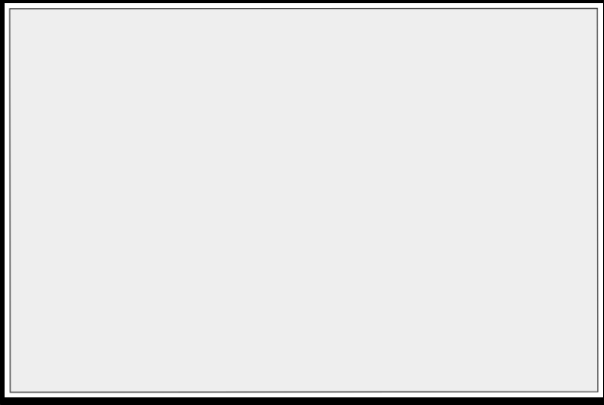
meters

Estimated Work Days

5

days (at ~15 sqm/day)

2D Blueprint



Downloads

Download OBJ (3D Model)

Download Blueprint (SVG)

Download OBJ (3D Model)

Download Blueprint (SVG)

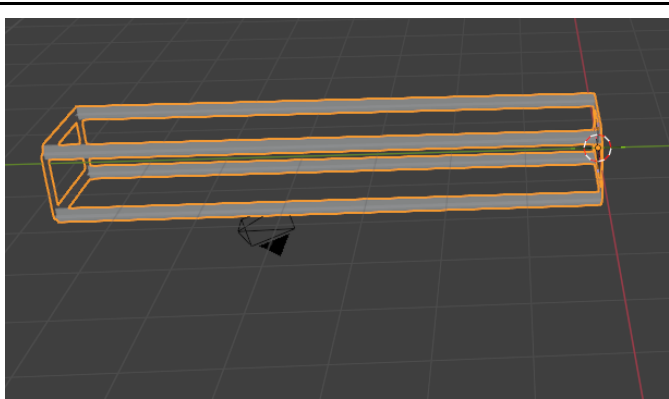
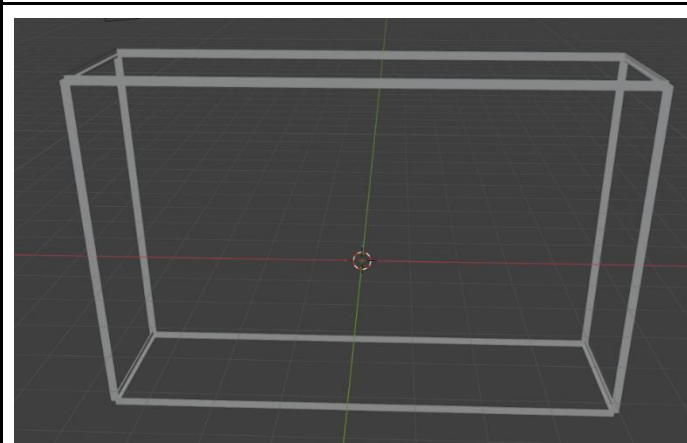
Vertex Coordinates (x, y, z)

Total vertices: 4

```
[
  [
    -5.74,
    3.73,
    0.0
  ],
  [
    -5.73,
    -3.74,
    0.0
  ],
  [
    5.74,
    3.73,
    0.0
  ],
  [
    5.73,
    -3.74,
    0.0
  ]
]
```

Plan Metadata

```
{
  "scale": 100.0,
  "height": 2.2,
  "input_file": "C:/Users/Navika/Desktop/fnp/sample_input/sample_blueprint_open_office.png"
}
```



CHAPTER 11

REFERENCES

1. Parivartan – 2D Blueprint to 3D Model. (2025). *International Journal of Research Publication and Reviews*, Vol. 6, Issue 3, pp. 5094–5097.
2. Venkat Krishna Reddy G. N., Prajwal Venu Gopal Reddy N., Srushti G. S., Nirmala S. (2025). *Plan–MORPH: Conversion of 2D Floor Plans to 3D Models*. *International Journal of Enhanced Research in Science, Technology & Engineering*.
3. “Converting 2D Floor Plans to 3D Models Using Computer Vision and Machine Learning.” (2025). *International Research Journal of Modernization in Engineering, Technology and Science (IRJMETS)*, January 2025 issue.
4. “From Blueprints to Reality: Converting 2D Plans into 3D Interactive Spaces.” (2024). *International Research Journal of Architecture and Engineering (IRJAEH)*, 12(3).
5. “Leveraging Machine Learning Techniques in Converting 2D Floor Plans to 3D Building Models.” (2025). *Procedia Computer Science*.
6. “Parivartan-2D Blueprint to 3D Model.” (2025). *International Journal of Research Publication and Reviews*, Vol. 6, Issue 3.
7. “2D to 3D Floor Plan Modeling using Image Processing and Augmented Reality.” (2023). *International Research Journal of Engineering and Technology (IRJET)*, Vol. 9, Issue 4, pp. 2609–2614.
8. Park, B., & Kim, C. (2021). *3DPlanNet: Generating 3D Models from 2D Floor Plan Images Using Ensemble Methods*. CVPR Workshop / related venue.
9. “Conversion of 2D Blueprints into 3D Models.” (2023). *International Journal for Innovative Research in Technology (IJIRT)*, Vol. 9, Issue 12, pp. 91–94.

10. “Multi-Unit Floor Plan Recognition and Reconstruction Using Improved Semantic Segmentation of Raster-Wise Floor Plans.” (2024).
(MDA-UNet / MACU-Net based methods for floor plan segmentation and 3D reconstruction).